Home assignment: build a small data warehouse for an ad network

Thank you for your interest in the Senior Engineer, Data and ML Platform role at Falcon Labs. This assignment is close to the real work: the same kind of data, the same kind of tables, the same questions we ask ourselves every week.

1. Your goal

You get raw data from our ad network: requests, impressions, clicks, conversions, orders, and the tables that describe publishers, sites, placements, advertisers, campaigns and banners. The data is real but masked: ids, names and money amounts are changed, personal data is removed; dates and times are real. Like any real data, it has problems. Finding and handling them is part of the job.

The data comes as **daily portions**, one folder per day, 63 days. This simulates a daily job that fetches new data from our production system (see section 4). Your pipeline must process the data day by day, in order, the way a scheduled job would.

Build, in PySpark:

1. **A staging layer.** Load every input file with an explicit schema. Keep the raw data as it is; fix nothing here.
2. **A data warehouse.** Dimension tables and fact tables with a clear grain. At minimum: a placement dimension (placement → site → publisher), a banner dimension (banner → campaign → advertiser), a request fact and an impression fact with click and conversion outcomes, one row per shown slot (fid; the raw impressions table can have several rows per slot).
3. **Data mart: ML training table.** One row per shown slot (fid), like the impression fact. Columns: labels (is_click, is_conversion), features that were known at the moment of the request (request context including the order amount in USD and the device type, OS and browser parsed from user_agent, placement and banner attributes, site and campaign enrichment, time features, holiday flag, and at least two history features such as the publisher's conversion rate over the previous 7 and 28 days). Nothing from the future may leak into a feature. For this exercise, take the supplied dimension and enrichment attributes as known on the day of their snapshot (see the note on dimension history below); enforce real arrival times for everything built from events, such as history features. The snapshots carry today's attribute values, so they cannot support a claim of a leakage-free historical backtest.
4. **Data quality report.** A table or file with checks and their results, per day: row counts in and out, duplicates, nulls in key columns, facts that do not match any dimension row, and anything else you find. For each problem: how many rows, an example, and what you did with them.
5. **External data.** Orders come in many currencies; the ML mart needs the order amount in USD. Get exchange rates from a public source (for example frankfurter.app). Add a public-holiday feature to the ML mart (for example the holidays package or Nager.Date). Both must be loaded by code, not pasted by hand.

Your job takes a date as a parameter and processes that day's folder on top of what was loaded before. Running the same day twice must not change the result. Loading a day must correctly update earlier days when late events arrive.

2. What we expect

- Clean PySpark. DataFrame API/SQL, built-in functions, window functions where they fit. UDFs only where there is no other way. No collect() on big data.
- A warehouse that someone else can understand: every table has a documented grain and key.
- Numbers that reconcile: impressions in the staging layer = impressions in the fact table + rows you rejected, and you can show it per day.
- Honest handling of bad data: report it, do not silently drop it with an inner join.
- A short README: how to run, what you built, what problems you found in the data, what you decided and why, what you would do differently with 100× more data.
- Tests for the parts that can go wrong (deduplication, fid decoding, user agent parsing, history features, currency conversion, re-running a day).

You may use AI tools. We will ask you to explain and change your code in the follow-up call, so make sure you understand everything you hand in.
Please spend about 4–6 hours. If you run out of time, write what is not finished and how you would do it. We prefer three things done well over six done halfway.

3. Execution environment

- Write the pipeline in PySpark and run it on local Spark instance or on a real Spark cluster (Databricks Free Edition is enough and costs nothing; any other cloud Spark - EMR, Dataproc, is also fine).
- Hand in the code as a Git repository (GitHub link or zip): notebooks or .py files, plus the run outputs (notebook with cell outputs, or logs).
- Output tables in Parquet or Delta, partitioned by date where it makes sense.
- Unit tests may run locally with a small SparkSession.
- Keys and secrets must not be in the code.

Your code must be fully reproducible: we are going to take your code and re-run on our Spark cluster. Therefore, please, don't hardcode cluster names or access parameters inside notebooks/py files: make a config file and programmatically read it, so that if we re-run it - we will just change the config.

4. Input dataset

What Falcon does: a shopper finishes an order in an online store (a **site** of a **publisher**). On the thank-you page we show offers (**banners**) of other brands (**advertisers**). One page view is a **request**. Each banner shown is an **impression**. The shopper may **click** and later buy from the advertiser; the advertiser reports this to us as a **postback** (a conversion) with revenue. We pay the publisher a share of the revenue.

Daily portions

Layout:

```
dataset/
  README.md          data dictionary: every table, every column
  manifest.json      row counts per table per day
  dt=2026-08-24/     day 1
  dt=2026-08-25/     day 2
  ...
  dt=2026-10-25/     day 63
```

Each folder dt=YYYY-MM-DD is **what our loader fetched from the production database at the end of that day**. We are simulating a daily job that pulls data from the source system.
Inside a folder:

Table kind	What the day folder contains
Event tables: requests, impressions, clicks, postbacks, orders	only the rows created on that day (UTC). Nothing from other days.
Dimension tables: placement, site, publisher, platform, category, banner, campaign, advertiser	a full copy of the table: every row that existed at the end of that day.
Enrichment tables: site_enrichment, campaign_enrichment	a full copy as of that day.

This means:

- A click or a postback sits in the folder of the day it **happened**, which can be days after the impression it belongs to. Day 10 can bring a conversion for an impression from day 2.
- A dimension row can change from one day to the next (for example a publisher's revenue share). You see the whole table every day and decide what to do with changes.
- The folders are what arrived. If something arrived twice, late, or not at all, that is what you get, as in real life.

Tables

Table	Format	One row is	Key	Links to
requests	JSON lines (gz)	one ad request (one order page view)	request_id	placement_id; data holds the order details as nested JSON
impressions	Parquet	one render of a banner in one slot of one request; a slot (fid) can have several rows	impression_id	banner_id; fid encodes the request and the slot position using base64

Table	Format	One row is	Key	Links to
clicks	Parquet	one click on a slot	click_id	fid
postbacks	Parquet	one conversion reported by an advertiser	postback_id	fid; has revenue
orders	CSV	one order on a publisher's site	order_id + site_id	site_id, placement_id, customer_id; amount in local currency
placement	CSV	one ad slot configuration on a site	id	site_id
site	CSV	one online store	id	publisher_id, category_id
publisher	CSV	one publisher (owner of sites)	id	platform_id; has rev_share_percentage
platform	CSV	e-commerce platform (Shopify etc.)	id	
category	CSV	category tree, one level of parent	id	parent_category_id
banner	CSV	one creative	id	campaign_id
campaign	CSV	one advertiser campaign, has CPA	id	advertiser_id, category_id
advertiser	CSV	one advertiser	id	
site_enrichment	JSON lines	third-party attributes of a site (audience, traffic, category), one row per fetch	site_id + created_at	site_id
campaign_enrichment	JSON lines	same, for a campaign	campaign_id + created_at	campaign_id

Notes:

- All timestamps are UTC unless the data says otherwise.
- fid is the link between impressions, clicks and postbacks. It is base64url of the string request_id:position, where position is the slot number starting at 0. Decode it to link an impression to its request.
- Enrichment tables may contain several rows for the same site or campaign. Use the newest one that existed at the time of the request.
- Our source database keeps no history of dimension attributes. A dimension snapshot therefore shows every row that existed on that day with the values it has today, and updated_at can be later than the folder date. Treat the snapshot of a day as what you knew on that day; where two snapshots differ, the later one wins from its day on.
- Revenue in postbacks is in USD. Order amounts are in the order's currency.
- Not every request has impressions. Not every postback has a matching impression. Not every id matches a dimension row. This is normal for us; we want to see how you deal with it.

5. What to hand in

1. Repository link or zip with the code, tests and README.
2. The run outputs (notebook with outputs or logs) for the full run over all 63 days.
3. Proof that re-running one day does not change the result (row counts and sums before and after).
4. The data quality report.
5. Row counts of every output table after the full run.